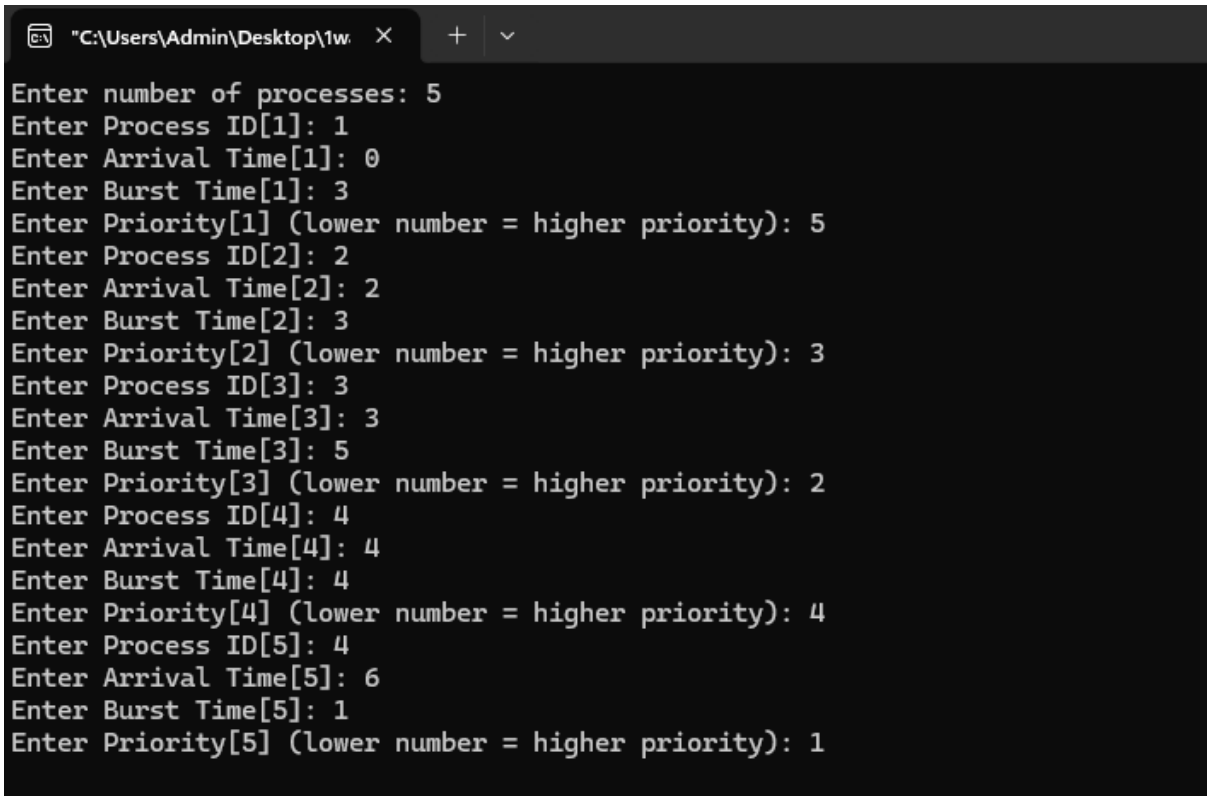
1)

```
#include <stdio.h>
#include <limits.h>

#define MAX_PROCESSES 100

int main() {
    int n;
    int Process_ID[MAX_PROCESSES];
    int Arrival_Time[MAX_PROCESSES];
    int Burst_Time[MAX_PROCESSES];
    int Priority[MAX_PROCESSES];
    int Completed[MAX_PROCESSES];
    int Completion_Time[MAX_PROCESSES];
    int Turnaround_Time[MAX_PROCESSES];
    int Waiting_Time[MAX_PROCESSES];

    int Current_Time = 0;
    int Completed_Count = 0;
    int i, Selected_Process;
    int Highest_Priority;

    // Input number of processes
    printf("Enter number of processes: ");
    scanf("%d", &n);

    // Input process details
    for (i = 0; i < n; i++) {
        printf("Enter Process ID[%d]: ", i+1);
        scanf("%d", &Process_ID[i]);
```

```
    printf("Enter Arrival Time[%d]: ", i+1);
    scanf("%d", &Arrival_Time[i]);
    printf("Enter Burst Time[%d]: ", i+1);
    scanf("%d", &Burst_Time[i]);
    printf("Enter Priority[%d] (lower number = higher priority): ", i+1);
    scanf("%d", &Priority[i]);
    Completed[i] = 0;  // false
}

// Priority Scheduling Algorithm
while (Completed_Count < n) {
    Highest_Priority = INT_MAX;
    Selected_Process = -1;

    // Find highest priority process that has arrived
    for (i = 0; i < n; i++) {
        if (Arrival_Time[i] <= Current_Time && Completed[i] == 0) {
            if (Priority[i] < Highest_Priority) {
                Highest_Priority = Priority[i];
                Selected_Process = i;
            }
        }
    }

    if (Selected_Process == -1) {
        // No process available, increment time
        Current_Time++;
    } else {
        // Execute selected process
        int Start_Time = Current_Time;
        Completion_Time[Selected_Process] = Start_Time + Burst_Time[Selected_Process];
```

```
        Turnaround_Time[Selected_Process] =
            Completion_Time[Selected_Process] - Arrival_Time[Selected_Process];

        Waiting_Time[Selected_Process] =
            Turnaround_Time[Selected_Process] - Burst_Time[Selected_Process];

        Current_Time = Completion_Time[Selected_Process];
        Completed[Selected_Process] = 1;  // true
        Completed_Count++;
    }
}

// Display results
printf("\nProcess Scheduling Results:\n");
printf("PID\tArrTime\tBurst\tPriority\tWait\tTurnaround\n");
printf("---\t------\t-----\t--------\t----\t---------\n");

for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t\t%d\t%d\n",
            Process_ID[i], Arrival_Time[i], Burst_Time[i],
            Priority[i], Waiting_Time[i], Turnaround_Time[i]);
}

// Calculate and display averages
float avg_wait = 0, avg_turn = 0;
for (i = 0; i < n; i++) {
    avg_wait += Waiting_Time[i];
    avg_turn += Turnaround_Time[i];
}
printf("\nAverage Waiting Time: %.2f\n", avg_wait / n);
```

```
    printf("Average Turnaround Time: %.2f\n", avg_turn / n);


    return 0;
}
```



```
Process Scheduling Results:
PID     ArrTime Burst   Priority        Wait    Turnaround
---     ------  -----   --------        ----    ---------
1       0       3       5               0       3
2       2       3       3               7       10
3       3       5       2               0       5
4       4       4       4               8       12
4       6       1       1               2       3

Average Waiting Time: 3.40
Average Turnaround Time: 6.60

Process returned 0 (0x0)   execution time : 93.338 s
Press any key to continue.
```

2)

```
#include <stdio.h>

#include <limits.h>


#define MAX 100


int main() {

    int n;

    int pid[MAX], at[MAX], bt[MAX], prio[MAX];

    int rt[MAX];              // Remaining time

    int ct[MAX], tat[MAX], wt[MAX];

    int current_time = 0;

    int completed = 0;

    int i;


    printf("Enter number of processes: ");

    scanf("%d", &n);


    // Input

    for (i = 0; i < n; i++) {
```

```
    printf("Enter Process ID, Arrival Time, Burst Time, Priority for process %d: ", i + 1);
    scanf("%d %d %d %d", &pid[i], &at[i], &bt[i], &prio[i]);
    rt[i] = bt[i];          // Remaining_Time[i] = Burst_Time[i]
}

// Preemptive Priority Scheduling
while (completed < n) {
    int highest_priority = INT_MAX;   // large_number equivalent
    int selected = -1;

    // Select process with highest priority among arrived processes
    for (i = 0; i < n; i++) {
        if (at[i] <= current_time && rt[i] > 0) {
            if (prio[i] < highest_priority) {
                highest_priority = prio[i];
                selected = i;
            }
        }
    }

    if (selected == -1) {
        // No process available at this time
        current_time++;
    } else {
        // Execute selected process for 1 unit (preemption possible each unit)
        rt[selected]--;
        current_time++;

        // If process finished
        if (rt[selected] == 0) {
            ct[selected] = current_time;
```

```
            tat[selected] = ct[selected] - at[selected];
            wt[selected]  = tat[selected] - bt[selected];
            completed++;
        }
    }
  }

  // Output
  printf("\nPID\tAT\tBT\tPR\tCT\tTAT\tWT\n");
  printf("----\t--\t--\t--\t---\t--\n");
  float avg_wt = 0, avg_tat = 0;
  for (i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
          pid[i], at[i], bt[i], prio[i], ct[i], tat[i], wt[i]);
    avg_wt  += wt[i];
    avg_tat += tat[i];
  }

  printf("\nAverage Waiting Time: %.2f\n", avg_wt / n);
  printf("Average Turnaround Time: %.2f\n", avg_tat / n);

  return 0;
}
```

```
C:\Users\Admin\Desktop\1wa

Enter number of processes: 3
Enter Process ID, Arrival Time, Burst Time, Priority for process 1: 2 3 4 5
Enter Process ID, Arrival Time, Burst Time, Priority for process 2: 6 7 4 3
Enter Process ID, Arrival Time, Burst Time, Priority for process 3: 9 8 3 4

PID     AT      BT      PR      CT      TAT     WT
----    --      --      --      --      ---     --
2       3       4       5       7       4       0
6       7       4       3       11      4       0
9       8       3       4       14      6       3

Average Waiting Time: 1.00
Average Turnaround Time: 4.67

Process returned 0 (0x0)   execution time : 28.574 s
Press any key to continue.
```

3)

```
#include <stdio.h>


#define MAX 100


int main() {
    int n, tq;
    int pid[MAX], at[MAX], bt[MAX], rt[MAX];
    int wt[MAX] = {0}, tat[MAX] = {0};
    int completed = 0;
    int current_time = 0;

    // Simple circular queue for ready queue (store indices of processes)
    int rq[MAX];
    int front = 0, rear = 0;   // queue is empty when front == rear

    printf("Enter number of processes: ");
    scanf("%d", &n);
```

```
printf("Enter time quantum: ");
scanf("%d", &tq);

for (int i = 0; i < n; i++) {
    printf("Enter PID, Arrival Time, Burst Time for process %d: ", i + 1);
    scanf("%d %d %d", &pid[i], &at[i], &bt[i]);
    rt[i] = bt[i];  // remaining_time[i] = burst_time[i]
}

// Function-like macro to check if queue is empty
#define QUEUE_EMPTY (front == rear)

// Add newly arrived processes into ready_queue
auto void add_new_arrivals() {
    for (int i = 0; i < n; i++) {
        // process not yet completed and has arrived
        if (at[i] <= current_time && rt[i] > 0) {
            // check if already in queue (simple scan)
            int in_queue = 0;
            for (int j = front; j != rear; j = (j + 1) % MAX) {
                if (rq[j] == i) {
                    in_queue = 1;
                    break;
                }
            }
            if (!in_queue) {
                rq[rear] = i;
                rear = (rear + 1) % MAX;
            }
        }
    }
```

```
}

// Main Round Robin loop
while (completed < n) {
    // Add all processes that have arrived up to current_time
    add_new_arrivals();

    if (QUEUE_EMPTY) {
        // CPU idle, move time forward
        current_time++;
        continue;
    }

    // Dequeue from front
    int idx = rq[front];
    front = (front + 1) % MAX;

    // Execute process for min(remaining_time, time_quantum)
    int exec_time = (rt[idx] > tq) ? tq : rt[idx];
    rt[idx] -= exec_time;
    current_time += exec_time;

    // After execution, add newly arrived processes
    add_new_arrivals();

    if (rt[idx] > 0) {
        // not finished, put back into end of queue
        rq[rear] = idx;
        rear = (rear + 1) % MAX;
    } else {
        // finished
```

```
            tat[idx] = current_time - at[idx];
            wt[idx]  = tat[idx] - bt[idx];
            if (wt[idx] < 0) wt[idx] = 0;  // safety
            completed++;
        }
    }

    // Output
    printf("\nPID\tAT\tBT\tWT\tTAT\n");
    printf("----\t--\t--\t--\t---\n");
    float avg_wt = 0, avg_tat = 0;
    for (int i = 0; i < n; i++) {
        printf("%d\t%d\t%d\t%d\t%d\n",
               pid[i], at[i], bt[i], wt[i], tat[i]);
        avg_wt  += wt[i];
        avg_tat += tat[i];
    }

    printf("\nAverage Waiting Time: %.2f\n", avg_wt / n);
    printf("Average Turnaround Time: %.2f\n", avg_tat / n);

    return 0;
}
```

```
Enter number of processes: 4
Enter time quantum: 2
Enter PID, Arrival Time, Burst Time for process 1: 4 7 8
Enter PID, Arrival Time, Burst Time for process 2: 8 5 9
Enter PID, Arrival Time, Burst Time for process 3: 9 8 6
Enter PID, Arrival Time, Burst Time for process 4: 3 5 7

PID     AT      BT      WT      TAT
----    --      --      --      ---
4       7       8       0       0
8       5       9       14      23
9       8       6       18      24
3       5       7       21      28

Average Waiting Time: 13.25
Average Turnaround Time: 18.75

Process returned 0 (0x0)   execution time : 18.359 s
Press any key to continue.
```